Lab 09

Trees for land and resource decisions

Teaching Week 09 | Individual work. Submit in R or Python.

Purpose

A decision tree asks one question at a time, and every question it adds fits the training rows better, so the training error alone can never say when to stop. You will grow a classification tree that sorts felled trees by species or peat cores by wetland class, check its first question by hand, watch it overfit, prune it with the cost-complexity criterion and the one-standard-error rule, and read the pruned tree as rules. A short regression tree on a numeric column closes the lab. The lab opens on November 5, 2026, with the class on pruning, and runs over Fall Break until November 19.

Learning outcomes

After completing this lab, you should be able to

- grow a classification tree under stated stopping rules, and check the Gini decrease of its first question from the class counts;
- read from training and held-out accuracy where a growing tree starts to overfit;
- prune by cost-complexity, choosing the size by cross-validation and the 1-SE rule, and judge the result once on held-out rows; and
- read a pruned tree as rules, and say what a leaf of a regression tree predicts.

Dataset choice

Choose one of the verified open datasets below. Each option supports the assessment when your analytical table satisfies the data contract in the next section. The expected variables describe the published record and may differ in detail from the extract you retrieve; reconcile any difference in your report.

Option A | Biomass of trees sampled across Canada as part of the Energy from the Forest Biomass (ENFOR) Program

Publisher. Natural Resources Canada
Official record. https://open.canada.ca/data/en/dataset/fbad665e-8ac9-4635-9f84-e4fd53a6253c
Record identifier. `fbad665e-8ac9-4635-9f84-e4fd53a6253c`
Licence and format. Open Government Licence – Canada; CSV with a DOC metadata file
Expected variables. province; year; location; plot; tree; species; diameter at breast height; height; oven-dry mass of the stem, stem wood, stem bark, crown, foliage and twigs, branches and whole tree; latitude; longitude

Use in this lab. classify a felled tree's species from its size and from how its biomass divides between wood, bark and crown, and predict its whole-tree mass
Why it fits. The file holds 9 454 trees under 49 species labels, felled between 1978 and 1983 in 11 provinces and territories, and two species are each spelled two ways. The six most common species hold 5 338 trees, black spruce 1 764, white spruce 932, trembling aspen 799, balsam fir 639, white birch 629 and tamarack 575. Stem wood and bark are missing for 477 Newfoundland trees. In Quebec, 1 174 trees of these six species carry a branch mass of 0 because the branches were weighed with the foliage, so use the crown mass rather than its two parts. The stem, crown and whole-tree masses are sums of their parts. Ontario and Quebec each appear under two codes, which the metadata explains. Each province was sampled by its own crew and corrected by its own rules, and the Yukon holds only three of the six species, so the province is a natural group that gives part of the species away.

Option B | Peat Profile Database

Publisher. Natural Resources Canada
Official record. https://open.canada.ca/data/en/dataset/e287b0dd-8b07-435b-8a76-0c7ec84ac228
Record identifier. `e287b0dd-8b07-435b-8a76-0c7ec84ac228`
Licence and format. Open Government Licence – Canada; CSV tables with a lookup table and references
Expected variables. core; source site; province; latitude; longitude; sampling year; wetland class, form and type; fen type; vegetation; peat depth; carbon stock; permafrost; treed; water table; sample depth; bulk density; ash; humification; peat material; carbon; pH; calcium, magnesium, sodium and potassium
Use in this lab. classify a peat core's wetland as a bog, a poor fen or a rich fen from its depth and its peat, and predict its peat depth
Why it fits. The core table holds 1 217 cores, 760 bogs, 392 fens, 24 swamps, one marsh and 40 with no class. The fen type is recorded for 259 fens, 68 poor and 191 moderately rich, rich or extremely rich, which gives three classes on 1 019 cores. The poor fens are so few that a pruned tree may never predict one, so read their recall. The profile table holds 37 072 depth samples, a median of 11 per core, so summarize each core first, for example by its mean bulk density and ash and the share of its samples whose main material is Sphagnum or herbaceous peat. Of the 1 019 cores, ash is missing for every sample of 202, the peat material for 108 and pH for 594. Bulk density is estimated rather than measured throughout 233 cores, and carbon throughout 621, partly from the peat material, so a carbon content restates the material. The wetland class and the fen type define the target, and the wetland form and type restate it, since every palsa and peat plateau here is a bog, so none of the four is a predictor. The cores come from 735 sites in 12 provinces and territories, up to seven per site, and all 230 New Brunswick cores are bogs, so the site and the province are natural groups.

Data contract

Choose one listed dataset and build one multiclass modelling table.

- Define one target with at least three classes, the species or the wetland class. Merge or drop rare classes by one stated rule, and report the class counts before and after.
- Retain at least 300 complete rows, one per tree or core.

- Keep at least four predictors, at least two of them numeric, each justified from the documentation. A share or a per-core mean you compute counts, such as a tree's crown mass over its whole mass or a core's mean bulk density. A category enters R as a factor and Python as indicator columns. Identifiers, province and site codes, latitude and longitude, and every column the option names as restating the target are not predictors.
- Hold out a stratified quarter of the rows as the test set, with a recorded seed, and save which rows it holds. Every choice is made on the other three quarters.
- Grow every classification tree with the Gini impurity, every regression tree with the squared error, and a smallest leaf of five rows, which is `minbucket = 5`, `minsplit = 10` and `cp = -1` in `rpart` and `min_samples_leaf=5` with a fixed `random_state` in scikit-learn. The one entropy refit of Task 2 is the exception to the Gini impurity. At `cp = 0` `rpart` still drops a split that lowers the Gini impurity but not the training errors, so its tree would be smaller than scikit-learn's.
- Report accuracy, balanced accuracy and the recall of every class, beside a baseline that always predicts the most common training class.
- Document the official record URL, publisher, licence, retrieval steps, and access date.

What the contract does not settle

Meeting the contract does not show that the pruned tree's questions are the ones that matter, since a tree grown on another sample may ask different ones, or that a score from one random split would hold in a province or site the model has never seen.

Selection checklist

Resolve every unchecked item before you begin the analysis.

- ☐ I chose one option from the dataset catalogue above.
- ☐ I recorded the official URL, publisher, licence, access date, and retrieval steps.
- ☐ I defined a target with at least three classes and stated the rule that merges or drops rare classes.
- ☐ My table satisfies every rule in the data contract.
- ☐ I kept identifiers, province and site codes, and every column that restates the target out of the predictors.
- ☐ I held out a stratified test set with a recorded seed, and I use it only to score.
- ☐ I set the same stopping rules for every tree, and I report them.

Cumulative method

This lab carries forward the earlier data audit, cleaning and joins, graphics, baselines, class-aware metrics, the holdout and cross-validation. Your report must show how those earlier decisions support the current analysis rather than repeating them as disconnected exercises.

Work program

1. Build the table and hold out a test set

State the target, the merge rule with the class counts before and after, and each predictor with its unit and the reason it should carry information about the class. If a row is built from several tables, say how it was joined or summarized and how many rows the join kept. Report how many rows each gap removes. Hold out a stratified quarter of the rows as the test set with a recorded seed, and report the class counts on each side. Every choice below is made on the training rows, and the test rows are only scored. Score a baseline that always predicts the most common training class on the test rows.

2. Grow the tree, check its first question and watch it overfit

Grow one tree on the training rows under the stopping rules of the contract, and report its leaves, its depth, and its accuracy on the training and the test rows. In R the depth is the largest `floor(log2(n))` over the node numbers `n` in `rownames(fit$frame)`. Name its root question. From the class counts of the root and of its two children, compute the Gini impurity of each box, then the decrease the question achieves, the root's impurity minus the children's impurities weighted by their shares of the rows, and check it against the fitted tree. In R, `summary()` reports the improvement as the number of rows times this decrease. In Python, `tree_.impurity` and `tree_.n_node_samples` hold the impurity and the rows of every node, and the root's children are `tree_.children_left[0]` and `tree_.children_right[0]`. From the same counts, compute the decrease in entropy the question achieves. Then refit the tree once with entropy, `parms = list(split = "information")` in `rpart` or `criterion="entropy"` in scikit-learn, and once on a seeded random 90% of the training rows, and say whether the root question or the two questions below it change. Then grow trees with a maximum depth of 1 to 10 under the same rules, and plot their training and test accuracy against depth. Say where the tree underfits, where the test curve stops rising, how far apart the two curves are at depth 10, and why the training curve alone can never say when to stop. The test rows describe this curve and choose nothing.

3. Prune it and score it once

Choose the size of the tree by five-fold cross-validation on the training rows and the 1-SE rule. In R, grow the tree of Task 2 with `xval = 5` after `set.seed()`, print its cp table with `printcp()`, keep the smallest tree whose `xerror` is at most the smallest `xerror` plus that row's `xstd`, and cut it with `prune()` at the `CP` of the row you keep. In Python, `cost_complexity_pruning_path()` returns the values of `ccp_alpha` but no error estimate, so score `DecisionTreeClassifier(min_samples_leaf=5, ccp_alpha=a)` for each value with `cross_val_score` on five stratified folds of the training rows, keep the largest value whose mean error, one minus the accuracy, is at most the smallest mean error plus the standard error at that smallest value, the sample standard deviation of its five fold errors over $\sqrt{5}$, and refit with it on all training rows. Report the sequence as a table of leaves, parameter, error and standard error, plot the error against the leaves with one standard error as bars, and mark the best tree and the one the rule keeps. `rpart` prices a leaf in misclassified rows divided by the root's, scikit-learn in Gini impurity weighted by each leaf's share of the rows, and the two compute the standard error differently, so the two languages need not keep the same tree. Then score the pruned tree on the test rows by accuracy, balanced accuracy and the recall of every class, set it beside the tree of Task 2 and the baseline, and give its confusion matrix. Say what pruning cost or gained, and which classes pay.

4. Read the rules and grow a regression tree

Draw the pruned tree and write its five largest leaves as rules a forester or a peatland manager could apply, or all of them if it keeps five or fewer, with the number of training rows that reach each and the

share of them in its predicted class. Name one rule the documentation leads you to expect and one it does not. Then grow and prune a regression tree in the same way for one numeric column, the whole-tree mass from the diameter and the height for option A, or the peat depth from the core's other predictors for option B, leaving out the carbon stock, which is computed from the depth, and the sample depths and thicknesses, which add up to it. Use `method = "anova"` in `rpart`, or `DecisionTreeRegressor` in scikit-learn scored with `scoring="neg_mean_squared_error"`, whose values you negate, on the same split. A number cannot be stratified, so cross-validate on five ordinary folds, `KFold(5, shuffle=True)` with a seed. Report only its leaves before and after pruning, its test root mean squared error beside that of predicting the training mean, and one leaf, what it predicts and why that prediction is the mean of the training rows that reach it. End with the natural groups as a limitation. Trees from one province or cores from one site share crews, methods and conditions, and a random split puts a test row's group mates in training. Say in which direction this is likely to move your test scores.

Reproducible workflow

You may complete and submit the lab in either R or Python. Both paths are marked against the same evidence and rubric, and both must run end to end from one documented entry point in your portfolio repository.

Write your own analysis using the methods taught in the course. Document how to run it from the retained source data and produce every result cited in your report. Choose and justify the variables, preparation steps, and analytical methods required by your question and this lab's data contract. Record the software environment and any randomness that affects reproducibility.

Submit your own R or Python source and the evidence supporting the report. Include `split.csv` with the side of every row, training or test; `depth-curve.csv` with the training and test accuracy of every depth; `pruning-path.csv` with the leaves, the complexity parameter, the cross-validated error and its standard error of every tree in the classification sequence; and `test-scores.csv` with the test scores of the baseline, the tree of Task 2, the pruned tree, and the regression tree beside the training mean. Include the drawn pruned tree with its rules. Record the seeds, the merge rule, the stopping rules and the retained rows.

Downloaded data and generated results are not course source, so they never go into the course repository. Keep the work and its evidence in your private work repository until the deadline has passed and the instructor opens sharing, then publish it in your public GitHub portfolio repository. Link rather than redistribute source data unless its licence and size permit it.

Submission package

Keep the lab private before its submission deadline, including cloud notebooks and answer-revealing links. A branch in a public repository is not private. Give the instructor access to your private work repository. First commit the completed report, source, and supporting evidence. Record that commit's full SHA and commit URL in `labs/lab-09/README.md`, then commit this README update separately. A commit cannot contain its own SHA. Push both commits to the default branch before 16:05 Halifax time on November 19, 2026, the first Class B after Fall Break. The deadline holds even when no class meets that day. The marker reads that README on the default branch and assesses the snapshot it names. The immutable commit or release URL must resolve to that exact version of `labs/lab-09/`. A branch URL or movable tag alone is not a fixed submission. The package includes

- a concise PDF report that answers the analytical question, together with the completed selection checklist;

- the complete R or Python source plus the generated tables, figures, and any summary statistics referenced in the report;
- a README carrying the submitted commit SHA and its commit URL, the official data URL, retrieval instructions, exact execution command, and environment; and
- no redistributed source data unless its licence and size permit it.

After the deadline and when the instructor opens sharing, publish the same submitted commit and its history in your public GitHub portfolio repository and present that version at the next class meeting, if one follows. Preserve the submitted SHA when merging. Commit regularly as you work. Do not amend, rebase, squash, delete, replace, force-push, or backdate recorded assessment history. Keep submitted commits reachable and make corrections in new commits. The pushed README record and the instructor-retained snapshot provide the independent assessment reference; Git author dates alone are not proof of timely submission.

Assessment standard

The course uses one rubric across all integrated labs. The best nine of the ten weekly lab results contribute 15% of the course grade.

Table 1. Integrated lab marking criteria.

Criterion	Weight (%)
Dataset provenance and suitability	15
Reuse of earlier methods	20
Current-week analytical method	35
Interpretation and limitations	20
Reproducibility and communication	10
Total	100

Academic integrity, private GitHub exceptions, private grading, and publication limits follow the course syllabus. Generative AI output is prohibited on integrated labs, so generated code, prose, analysis, or figures cannot be submitted for lab credit. Cite the dataset and every non-trivial external source; the submitted analysis must be your own work.